CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 2

NAME: Kanishk Jamwal	**UID: 24BAI70827.**
BRANCH: CSE-AIML (CS221)	**SECTION/GROUP:** 24AIT_NTPP3[A]
SEMESTER: 5TH	**DATE OF PERFORMANCE:** 09/07/2026
SUBJECT: Full Stack - II	**SUBJECT CODE:** 24CSP-337

EXPERIMENT 2.1

1. AIM

To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data.

2. OBJECTIVE

1. To understand the concept of global state management
2. To implement Redux Toolkit for scalable state handling
3. To design a normalized state structure
4. To manage asynchronous data flows (optionally via mock APIs)

3. SOFTWARE REQUIREMENT

1. Node.js & npm
2. React.js
3. Redux Toolkit
4. React-Redux
5. Code Editor (VS Code)

4. THEORY

As frontend applications grow in complexity, managing state across multiple components becomes increasingly challenging. Traditional approaches such as prop drilling lead to tightly coupled components and reduced maintainability.

Redux provides a centralized store that acts as a single source of truth, enabling predictable state transitions through actions and reducers. Redux Toolkit simplifies Redux implementation by reducing boilerplate and providing built-in utilities for common tasks.

State normalization is an important concept where data is stored in a structured format (similar to relational databases), reducing redundancy and improving performance. This is especially useful when managing collections such as posts or users.

Additionally, Redux supports handling asynchronous operations, enabling integration with APIs or simulated mock data sources.

5. IMPLEMENTATION/PROCEDURE

1. Open Visual Studio Code.
2. Create a React application using Create React App.
3. Install Redux Toolkit and React-Redux libraries.
4. Configure the Redux Store using configureStore().

5. Create a Redux Slice (postsSlice.js) with an initial state.
6. Define reducers for Add, Update, and Delete operations.
7. Wrap the application with the Redux Provider.
8. Create the PostList component.
9. Use useSelector() to retrieve posts from the Redux Store.

10. Use useDispatch() to dispatch Redux actions.



CHANDIGARH
UNIVERSITY

Discover. Learn. Empower.

6.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

11. Implement CRUD operations: Add Post, Update Post, Delete Post
12. Design the interface using CSS.
13. Run the application using: npm start
14. Verify all CRUD operations.

CODE

Store. Js

```
import { configureStore } from "@reduxjs/toolkit";
import postsReducer from "../features/posts/postsSlice";

export const store = configureStore({
reducer: {  posts: postsReducer,
  },
});
```

PostsSlice. Js

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
items: [],
};

const postsSlice = createSlice({  name:
    "posts",                initialState,
reducers: {  addPost: (state, action)
=>                              {
state.items.push(action.payload);
  },

  updatePost: (state, action) => {
const { id, title } = action.payload;

   const post = state.items.find((post) => post.id === id);

   if    (post)    {
post.title = title;
   }
  },
```

```
  deletePost:      (state,    action)   =>
            {      state.items = state.items.filter(
(post) => post.id !== action.payload
     );
   },
  },
});
```

```
export const { addPost, updatePost, deletePost } = postsSlice.actions;
```

```
export default postsSlice.reducer;
```

Index. Js

```
import React from "react"; import
ReactDOM from "react-dom/client"; import
"./index.css";  import App from
"./App";

import { Provider } from "react-redux";  import { store }
from "./app/store";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

App. Js

```
import "./App.css";  import PostList from
        "./components/PostList";

function App() {
return (
   <div className="App">
<h1>Redux Content Manager</h1>

     <PostList />
   </div>
  );
}

export default App;
```

postList.js

```
import React, { useState } from "react"; import { useSelector,
        useDispatch } from "react-redux"; import {   addPost,   updatePost,
        deletePost,
} from "../features/posts/postsSlice";

function PostList() {    const [title, setTitle] =
useState("");
```

```
  const posts = useSelector((state) => state.posts.items);
const dispatch = useDispatch();

  const handleAddPost = () => {
if (title.trim() === "") return;

    dispatch(
addPost({         id:
Date.now(),         title:
title,
      })
    );

    setTitle("");
  };

  const handleUpdatePost = (post) => {      const
newTitle = prompt("Enter new title:", post.title);

    if (newTitle) {
dispatch(
updatePost({
id: post.id,
title: newTitle,
        })
      );
    }
  };

  const handleDeletePost = (id) => {
dispatch(deletePost(id));
  };

  return (
    <div className="container">

      <h1>🗄 Redux Content Manager</h1>

      <div className="input-section">
        <input          type="text"
placeholder="Enter Post Title..."
value={title}          onChange={(e) =>
setTitle(e.target.value)}
        />
        <button onClick={handleAddPost}>Add Post</button>
      </div>
```

```
        {posts.length === 0 ? (
          <p>No Posts Available</p>
        ) : (
          posts.map((post) => (
            <div className="card" key={post.id}>
<span>{post.title}</span>

              <div>
                <button              className="edit-btn"
onClick={() => handleUpdatePost(post)}
                >
                  Edit
</button>

                <button              className="delete-btn"
onClick={() => handleDeletePost(post.id)}
                >
                  Delete
                </button>
              </div>
            </div>
          ))
        )}
      </div>
    );
}

export default PostList;
```

App.cs body { margin: 0; background: linear-gradient(135deg, #667eea, #764ba2); font-family: Arial, sans-serif; }

.container { width: 700px; margin: 40px auto; background: white; padding: 30px; border-radius: 15px; box-shadow: 0 10px 25px rgba(0,0,0,0.2); }

h1 { text-align: center; color:

```
#4a148c;
}

.input-section {
display: flex; gap: 10px;
margin-bottom:
25px;
}

input { flex: 1; padding:
12px; border-radius: 8px;
border: 2px solid #667eea;
font-size: 16px;
}

button { padding:
10px 18px; border:
none; border-radius:
8px; cursor: pointer;
color: white;
fontweight: bold;
}

button:hover {
opacity: 0.9;
}

.input-section button {
background: #4caf50;
}

.card { display: flex; justify-content:
space-between; align-items: center;
background: #f4f6ff; margin: 12px 0;
padding: 15px; border-radius: 10px;
box-shadow: 0 3px 8px rgba(0,0,0,0.1);
}

.edit-btn { background:
#ff9800; margin-right: 10px;
}

.delete-btn {
background: #f44336; }
```

```
        p {    text-align:
        center;
        color: #666;
}
```

7. OUTPUT

The Redux Content Manager application was successfully implemented. Users were able to add new posts, edit existing posts, and delete posts through a centralized Redux Store. The interface updated automatically after every CRUD operation, demonstrating efficient state management using Redux Toolkit.



Fig: 2.1 Redux Content Manager application

8. LEARNING OUTCOMES

1. Understand derived state using Redux Selectors.
2. Implement memoized selectors using createSelector().
3. Optimize state retrieval from the Redux Store.
4. Reduce unnecessary recomputations using memoization.
5. Filter and search data efficiently.
6. Improve rendering performance in React applications.
7. Develop scalable and performance-optimized frontend applications using Redux Toolkit.



CHANDIGARH
UNIVERSITY

Discover. Learn. Empower.

1. CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT 2.2 AIM

To optimize state access and improve application performance using memoized selectors and efficient rendering strategies

2.

OBJECTIVE

1. To understand the concept of derived state
2. To implement memoized selectors using Reselect
3. To reduce unnecessary component re-renders 4. To improve performance in large-scale applications

3.

SOFTWARE REQUIREMENT

1. React.js
2. Redux Toolkit
3. Reselect (or built-in createSelector)
4. Code Editor **4.**

THEORY

In large-scale applications, inefficient state access and frequent re-renders can significantly degrade performance. Redux selectors are used to extract specific pieces of state; however, without optimization, they may recompute values unnecessarily.

Memoization is a technique used to cache computed results and return them only when input data changes. Libraries like Reselect provide memoized selectors that prevent redundant computations and reduce rendering overhead.

Derived state refers to data computed from the existing state rather than stored directly. By using selectors, applications can compute complex data efficiently without duplicating state.

Optimizing selectors and rendering behavior ensures that components update only when necessary, improving responsiveness and scalability.

5.

IMPLEMENTATION/PROCEDURE

1. Create a new React application.
2. Install Redux Toolkit and React-Redux.
3. Configure the Redux Store.
4. Create a Posts Slice containing sample post data.
5. Create memoized selectors using createSelector().
6. Create selectors for:Total Posts Short Posts Long Posts
7. Create a Dashboard component.
8. Display all posts using useSelector().
9. Implement a search bar for filtering posts.
10. Implement filter buttons:All, Short, Long 11. Display statistics using memoized selectors.
12. Apply CSS styling for the dashboard.
13. Run the application using: npm start

6. 14. Verify selector functionality and filtering



CHANDIGARH
UNIVERSITY

Discover. Learn. Empower.

CODE

Store. Js import { configureStore } from "@reduxjs/toolkit";

import postsReducer from "../features/posts/postsSlice";

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

```
export const store = configureStore({
reducer: {     posts: postsReducer,
  },
});
```

PostsSlice. Js import { createSlice } from "@reduxjs/toolkit";

```
const initialState = {
items: [
    { id: 1, title: "React Basics" },
    { id: 2, title: "Redux Toolkit" },
    { id: 3, title: "JavaScript ES6" },
    { id: 4, title: "Node.js" },
    { id: 5, title: "MongoDB" },
    { id: 6, title: "Express.js" },
    { id: 7, title: "HTML & CSS" },
    { id: 8, title: "Full Stack Development" },
  ],
};

const postsSlice = createSlice({   name: "posts",
initialState,   reducers: {},
});

export default postsSlice.reducer;
```

Selector.js import { createSelector } from "@reduxjs/toolkit";

```
// Basic Selector  export const selectPosts = (state) =>
state.posts.items;

// Memoized Selector - Total Posts export const
selectTotalPosts = createSelector(
  [selectPosts],
  (posts) => posts.length
);
// Memoized Selector - Short Posts (Title length < 12) export const
selectShortPosts = createSelector(
  [selectPosts],
  (posts) => posts.filter((post) => post.title.length < 12)
);
//
M
                e
                m
```

oi
z
e
d
S
el
e
ct
or
-
L
o
n
g
P
o
st
s
(
Ti
tl
e
le
n
gt
h
>
=
1
2)
e
x
p
or
t
c
o
n
st
se
le
ct

L

```
o
n
g
P
o
st
s
=
cr
e
at e S el e
ct or
(
  [selectPosts],
  (posts) => posts.filter((post) => post.title.length >= 12)
);
```

Dashboard.js

```
import React, { useState } from "react";  import { useSelector } from "react-redux";

import {
selectPosts,   selectTotalPosts,
selectShortPosts,   selectLongPosts,
} from "../features/posts/selectors";

function Dashboard() {   const posts =
useSelector(selectPosts);   const totalPosts =
useSelector(selectTotalPosts);   const shortPosts =
useSelector(selectShortPosts);    const longPosts =
useSelector(selectLongPosts);

  const [filter, setFilter] = useState("all");    const [search,
setSearch] = useState("");

  let filteredPosts = posts;

  if (filter === "short") {
filteredPosts = shortPosts;
  }

  if (filter === "long") {
filteredPosts = longPosts;
  }

  filteredPosts = filteredPosts.filter((post) =>
post.title.toLowerCase().includes(search.toLowerCase())
  );

  return (
```

```
<div className="dashboard">
```

```
      <h1>📊 Redux Selector Dashboard</h1>

      <input type="text" placeholder="🔍 Search
Posts..." value={search} onChange={(e) =>
setSearch(e.target.value)}
      />
      <div className="stats">

        <div className="card blue">
          <h2>{totalPosts}</h2>
          <p>Total Posts</p>
        </div>

        <div className="card green">
          <h2>{shortPosts.length}</h2>
          <p>Short Posts</p>
        </div>

        <div className="card orange">
          <h2>{longPosts.length}</h2>
          <p>Long Posts</p>
        </div>

      </div>

      <div className="buttons">
        <button onClick={() => setFilter("all")}>All</button>

        <button onClick={() => setFilter("short")}>
          Short
        </button>

        <button onClick={() => setFilter("long")}>
          Long
        </button>
</div>

      <div className="list">

        {filteredPosts.map((post) => (
          <div key={post.id} className="post">
            {post.title}
```

```
</div>
```

```
))}
      </div>

    </div>
  );
}

export default Dashboard;
```

App. Js

```
import "./App.css";  import Dashboard from
          "./components/Dashboard";

function App() {
return (
   <div className="App">
     <Dashboard />
   </div>
  );
}

export default App;
```

App.css

```
* {  margin: 0;  padding: 0;  box-
      sizing:  border-box;       fontfamily:
      "Segoe UI", sans-serif;
}

body {
  background: linear-gradient(135deg, #0f172a, #2563eb);  min-
height: 100vh;
}

.dashboard {  width:
90%;  max-width:
1000px;    margin: 40px auto;    background: #ffffff;
border-radius: 18px;  padding: 30px;    box-shadow:
0 15px 35px rgba(0,0,0,0.25);
}

.dashboard h1 {  textalign: center;
color:
#1e3a8a;  margin-
bottom: 25px;
}
```

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

```
.dashboard input {  width: 100%;
padding: 14px;  border: 2px solid #2563eb;
border-radius:  10px;
outline:  none;  font-size:  16px;
marginbottom: 25px;
}

.dashboard input:focus {  border-color: #1d4ed8;  box-
shadow: 0 0 8px
rgba(37,99,235,.3);
}

.stats {  display: flex;  gap: 20px;  justify-
content: spacebetween;  margin-bottom:
25px;
}

.card {  flex:  1;  padding:
20px;  border-radius:  15px;
color:  white;  textalign:
center;  transition: 0.3s;
}

.card:hover {  transform: translateY(-5px);
}

.card h2 {  font-size:
34px;  margin-
bottom: 8px;
}

.blue {  background: lineargradient(135deg,#2563eb,#1d4ed8);
}

.green {  background: lineargradient(135deg,#10b981,#059669);
}

.orange {  background: lineargradient(135deg,#f59e0b,#ea580c);
}
```

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

```
.buttons { display:
    flex; justify-content:
center; gap: 15px;
marginbottom: 30px;
}

.buttons button { background:
#2563eb; color: white; border:
none; padding: 12px 28px;
border-radius: 10px; cursor:
pointer; font-size: 15px;
fontweight: bold; transition:
.3s;
}

.buttons button:hover { background:
#1d4ed8; transform: scale(1.05);
}
 .list { display: flex; flex-
direction: column; gap: 15px;
}

.post { background: #f8fafc; border-
left: 6px solid #2563eb; padding: 18px;
border-radius: 12px; font-size: 18px;
fontweight: 500; transition: .3s;
}

.post:hover { background: #eff6ff;
                transform:
translateX(5px);
}

@media (max-width: 768px) {

  .stats { flex-direction:
column;
  } .b utt on
```

s { fle

```
xdir ect io n:
co lu m
n;
  }
  .buttons button {     width: 100%;
  }
}
```

7. OUTPUT

The Redux Selector Dashboard was successfully implemented using memoized selectors. The application displayed statistics such as total, short, and long posts, while allowing users to search and filter posts efficiently. Memoized selectors optimized state access and reduced unnecessary recomputations, resulting in improved application performance.

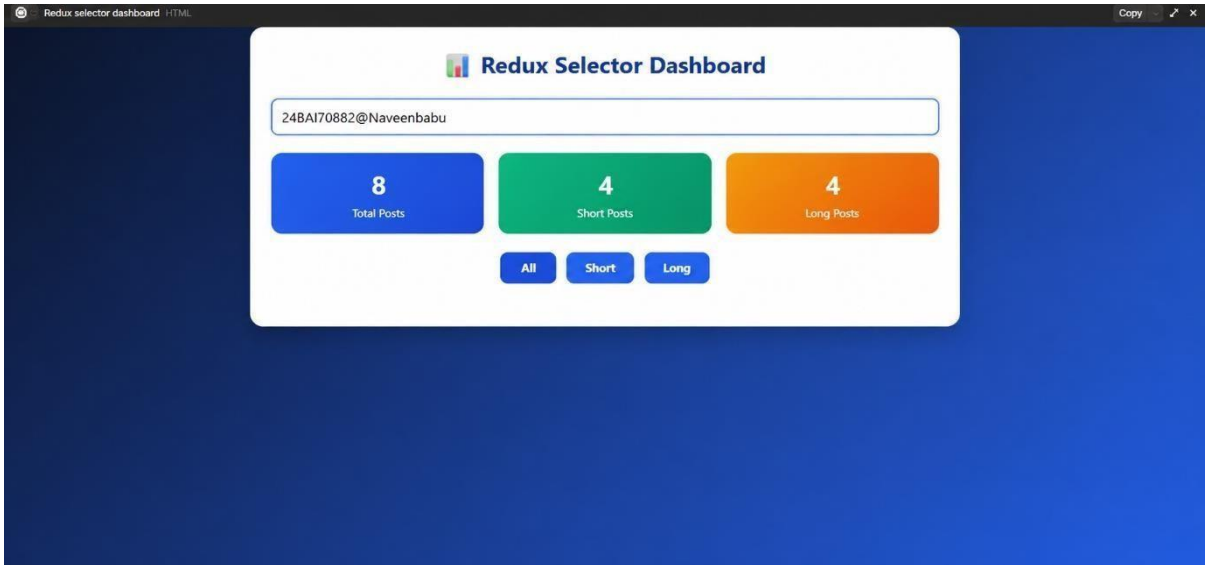


Fig: 2.2 Redux Selector Dashboard

8. LEARNING OUTCOMES

- Understand derived state using Redux Selectors.
- Implement memoized selectors using createSelector().
- Optimize state retrieval from the Redux Store.
- Reduce unnecessary recomputations using memoization.
- Filter and search data efficiently.
- Improve rendering performance in React applications.

Develop scalable and performance-optimized frontend applications using Redux Toolkit.